Resource Alchemy Hacker - Bug & Feature Tracker

Known Bugs

1. **Initial Layout Scaling Issue**
    - **Symptom:** UI elements (main active area, toolbar, status bar) do not automatically scale or refresh to fit the window size on launch. Only fixes itself after manual window resizing.
    - **Cause:** `WM_SIZE` event is likely not propagating correctly to all child controls (like `g_hwndInset`, `g_hwndStatus`) *after* they are fully initialized during `WM_CREATE` or in `WinMain`.
    - **Planned Fix:** Explicitly post or send a `WM_SIZE` message to the main window handle immediately after all UI initialization is complete in `WinMain`, forcing a complete layout recalculation.
2. **TreeView Icon Previews**
    - **Symptom:** Navigation tree fails to show individual previews for Icons or Icon Groups. It displays the custom root node icon for all child items instead of their actual icon.
    - **Planned Fix:** Update `GuiListResources` or TreeView image list logic to dynamically extract and assign the correct `HBITMAP`/`HICON` indices to child nodes corresponding to `RT_ICON` and `RT_GROUP_ICON` resources.
3. **"Version Info" Text Rendering & Word Wrap**
    - **Symptom:** "Version Info" text sometimes shows broken mojibake characters (e.g., "盲"?"). Text wraps between lines, which breaks syntax for scripts. No line numbers.
    - **Cause:** Encoding mismatch (e.g., trying to render UTF-16LE data as ASCII/ANSI, or misaligned byte length). Edit control is missing `WS_HSCROLL` and `ES_AUTOHSCROLL` window styles.
    - **Planned Fix:** Force proper Unicode string conversion for Version Info resource data. Add horizontal scroll styles to the multi-line edit control, and implement a custom side-panel or gutter for line numbers.
4. **"Extract Monolithic" Behavior**
    - **Symptom:** Currently saves a broken DLL/EXE instead of extracting all resources to folders.
    - **Planned Fix:** Rewrite the "Extract Monolithic" action to iterate over all resources, create a structured folder hierarchy (e.g., `OutputDir/Icon Groups/`, `OutputDir/Bitmaps/`), and dump each resource as its native file type. Ignore the raw "Icons" section to avoid duplication, relying solely on "Icon Groups".
5. **Right-Click Node Extraction (Context Menu)**
    - **Symptom:** Missing ability to right-click a specific tree node (e.g., "Bitmaps") and extract all items under that specific node type.

- **Planned Fix:** Add a context menu handler for the TreeView (`WM_CONTEXTMENU`), detect the node type, and iterate its children to extract them all into a user-specified folder.

6. **Action Menu Items Non-Functional**
   - **Symptom:** Menu options in the "Action" menubar don't do anything or are confusingly separated by type.
   - **Planned Fix:** Wire up the `WM_COMMAND` handlers for the Action menu to intelligently detect the currently selected resource in the TreeView, contextually applying the action (Replace/Extract) without needing separate confusing entries.
7. **"Add from resource script res / rc" Broken**
   - **Symptom:** Menu option does nothing.
   - **Planned Fix:** Implement the file dialog and CLI backend invocation to parse and compile `.rc`/`.res` files and inject them into the PE.
8. **Settings Dialog Aesthetic Clipping**
   - **Symptom:** Checkboxes and fields clip or are poorly positioned. Missing the matching title banner color gradient.
   - **Planned Fix:** Adjust `SettingsDlgProc` dialog resources and apply the same `WM_PAINT` banner gradient logic used in the main window.
9. **Drag and Drop Missing Functionality**
   - **Symptom:** Dragging a DLL onto the GUI executable only opens the GUI but doesn't load the file.
   - **Planned Fix:** Parse command-line arguments (`__argc`, `__argv` / `lpCmdLine`) in `WinMain`. If an argument is a valid PE file path, automatically call the open file routine during startup.
10. **Target File Corruption by Past AI Agent**
    - **Symptom:** `main.cpp` was recently overwritten with JSON log transcripts by an AI agent resulting in severe codebase corruption.
    - **Planned Fix:** Fully restore `main.cpp` using local diff patches or git history before applying further logic fixes.

Planned Improvements & New Features

1. **Auto-Backup System (`.bak` iteration)**
   - When saving, rename the original file to `.1.bak`, `.2.bak`, counting upwards automatically to prevent data loss.
2. **File Browser Style Viewer Tab**

- A new view mode to see icon-sized previews of a selected tree node in a grid (similar to Windows Explorer large icons view), rather than just a list.

3. **"Demo File!" Update**
   - Rename "Test Imageres" button to "Demo File!".
   - Target `C:\Windows\SystemResources\shell32.dll.mun` (or `shell32.mun`).
   - Automatically copy it to the working directory before opening to prevent system file corruption.
4. **Intelligent Image Conversion on Replace**
   - When replacing an image asset, automatically convert formats (PNG <-> ICO, BMP, JPG).
   - If a PNG is provided for an Icon Group, auto-scale to multiple sizes (16, 24, 32, 48, 64, 96, 128, 256) and build an ICO container dynamically.
   - Attempt background removal (pink, gray, black) for imported BMPs.
5. **Configurable Tree Icon Sizes**
   - Add options in the Settings dialog to toggle TreeView icon sizes between 16x16, 24x24, and 32x32.
6. **State Persistence**
   - Remember window position, dimensions, and maintain a history of the last 30 opened files via Registry or config file.
7. **Interactive Dialog Previews**
   - Enable clicking a button or double-clicking a dialog resource to render and preview the actual Win32 Dialog box.

Resource Alchemy Hacker

EliteSoftwareTech Co. - Antigravity Suite

Description

Resource Alchemy Hacker is a native C++ application for viewing, extracting, and replacing resources in Windows executables (PE files). It serves as a modernized clone of Resource Hacker and includes three main components:

1. **CLI Engine (`ResourceAlchemyHacker_CLI`)**: The backend worker executable that performs all resource extraction and injection using Win32 API.

2. **GUI Application (`ResourceAlchemyHacker_GUI`)**: The frontend Win32 application adhering to EliteSoftware GUI guidelines (Native Win32 aesthetics, Client Edge, Chin, 3D inset, etc.).
3. **Shell Extension (`ResourceAlchemyHacker_ShellExt`)**: A native C++ COM Shell Extension. It provides:
    - Cascading context menus for quick extraction/viewing.
    - A property sheet tab for executable files that lists resources (e.g., icons) like the native icon picker, and allows on-the-fly replacement of resources in system files (using backup renaming strategies).

Project Structure

- `/ResourceAlchemyHacker_CLI/` - Backend command line tool
- `/ResourceAlchemyHacker_GUI/` - Main Win32 application
- `/ResourceAlchemyHacker_ShellExt/` - COM Shell Extension (replaces old SharpShell implementation)
- `/Installer/` - InnoSetup script for deployment

Development Guidelines

- Strictly follows EliteSoftwareTech Co. guidelines for native Win32/WinForms aesthetics.
- Visual Styles must always be explicitly enabled.
- Title banner and 3D Inset Area.
- Dynamic Icon Targeting.

Features (v1.4.0.0 Update)

- **Advanced Resource Previews:**
    - Full Image rendering (Icons, Bitmaps, Cursors, Cursor Groups, Icon Groups).
    - Native AVI and WAV audio playback seamlessly integrated into the GUI.
    - Interactive Text Previews for Manifests, String Tables, Version Info, and Dialogs.
- **Resource Replacement & Extraction:**
    - Intuitive "Replace Resource" dialog with side-by-side graphical previews of original and new resources.
    - Seamless extraction and deletion of resources.
    - Accelerated hotkeys (Ctrl+R) and Context Menu support within the TreeView.
- **Enhanced UI & Aesthetics:**
    - Deep system-level icons dynamically loaded for all specific resource types in the TreeView (Audio, Images, XML, Manifests, Menus).
    - Legacy horizontal scrollbars hidden for a cleaner look (`TVS_NOHSCROLL`).
    - Strict adherence to EliteSoftwareTech Co. guidelines (Client Edge, 3D inset).
    - Registry-based persistence of Window dimensions and settings across sessions.
- **Full Menu Parity:**
    - 25+ classic Resource Hacker functions accessible via native Win32 dropdown menus (`File`, `Edit`, `View`, `Action`, `Help`).

Detailed Component Documentation

1. CLI Engine (`ResourceAlchemyHacker_CLI`)

The CLI is a headless command-line tool that performs resource modifications on PE files. It is designed to be called programmatically by scripts, build tools, or the GUI.

Syntax: `ResourceAlchemyHacker_CLI.exe <action> <target_file> [type] [name] [lang] [file]`

Actions:

- `/list`: Outputs a structured listing of all resources in `<target_file>`.
  - Example: `ResourceAlchemyHacker_CLI.exe /list C:\app.exe`
- `/extract`: Extracts a specific resource to an output file.
  - Example: `ResourceAlchemyHacker_CLI.exe /extract C:\app.exe #14 #1 1033 out.ico`
- `/replace`: Replaces an existing resource with data from `<file>`.
  - Example: `ResourceAlchemyHacker_CLI.exe /replace C:\app.exe #14 #1 1033 new_icon.ico`
- `/add`: Adds a new resource from `<file>`.
  - Example: `ResourceAlchemyHacker_CLI.exe /add C:\app.exe #2 #100 1033 image.bmp`
- `/delete`: Deletes a specific resource.
  - Example: `ResourceAlchemyHacker_CLI.exe /delete C:\app.exe #2 #100 1033`

Note: Types and Names can be strings (e.g. `ICON`) or integers prefixed with `#` (e.g. `#14` for RT_GROUP_ICON). Language should be a numeric LCID (e.g. 1033 for English-US).

2. Shell Extension (`ResourceAlchemyHacker_ShellExt`)

The Shell Extension integrates Resource Alchemy Hacker directly into Windows Explorer using a native C++ COM implementation.

- **Context Menus:** Right-clicking any PE file (.exe, .dll, .sys, etc.) provides quick shortcuts to Extract All Resources or Open in Resource Alchemy Hacker.
- **Property Sheet Tab:** Right-clicking a PE file and selecting "Properties" displays a new "Resources" tab.
  - The tab lists resources in a graphical grid (similar to the Windows icon picker).
  - Users can view and extract resources directly from the properties window without opening the full application.

Important Note on MUI and MUN Files

Modern Windows (Vista and later) frequently uses Multilingual User Interface (MUI) files and MUN files (in `C:\Windows\SystemResources`) to store localized resources (like icons, strings, and dialogs) separate from the main binary.

- When opening a system DLL (e.g. `imageres.dll`), the GUI will display resources dynamically loaded from its associated `.mui` or `.mun` file.
- **Warning:** Saving modifications to the original `.dll` file using this tool **will only write to the `.dll`**. It will **not** modify the `.mui` or `.mun` file. Because of strict core Windows `UpdateResource` API limitations, attempting to modify an icon or resource on a "stub" DLL that forwards to an MUI/MUN file may fail with **Error 50 (ERROR_NOT_SUPPORTED)**.
- **Solution:** To effectively modify system resources and avoid "Error 50", you must explicitly open and edit the actual `.mun` or `.mui` file directly (e.g., `C:\Windows\SystemResources\imageres.dll.mun`), rather than attempting to edit the base stub DLL.

Version

Document Version: 1.3.1.0

Gemini Operational Rules & Project Architecture

This file tracks specific instructions, workflow behaviors, and architectural overviews that Gemini (and other AI assistants) must follow when working on the `Local_APK_Store` project. This ensures a smooth development process and provides a clear map of how the ecosystem functions.

1. Core Operational Rules

Version Control & History (Auto-Pushing)

- **Rule:** Every time a file is modified, created, or any change is applied to the project, Gemini **must** immediately `git add .`, `git commit -m "..."`, and `git push -u origin master` to push the changes to `origin`.
- **Reasoning:** A constant, unbroken change history is required so the user doesn't need to manually worry about pushing, and we have a reliable log of all actions regardless of whether the specific code actually worked on the first try.

Line Endings (CRLF)

- **Rule:** Ensure all source files use `CRLF` (Windows style) line endings.
- **Reasoning:** Standardization across the Windows-based EliteSoftware ecosystem. Enforced via `.gitattributes`.

Continuous Changelog

- **Rule:** A `changelog.md` file must be continuously updated whenever things are changed or added to the project.
- **Reasoning:** Provides a human-readable, centralized history of new features, bug fixes, and architectural changes over time.

Elite App Marketplace Scope & Signatures

- **Rule (App Name):** The Android Client must strictly be named **"Elite App Marketplace"**.
- **Rule (APK Signing):** All processed APKs, including the Elite App Marketplace APK itself, must be signed using the master certificate located at `C:\Users\Administrator\Desktop\Local_APK_Store\Elite-EasySigner\EliteSoftware_Special.pfx` (Password: `Minecraft145!!`).
- **Rule (Client Features):** The marketplace client must support categorization, tagging, user reviews/comments, and the ability to download the `EliteSoftware_Special.cer` root certificate directly to the Android device via its internal Settings menu.

---

2. Project Architecture & Components

The project is split into two primary components: the **Windows Server Manager** and the **Android Client**.

A. Windows Server Manager (`/Manager_App/`)

This is a monolithic C++ Win32 desktop application that acts as both a GUI management tool and an HTTP server for the Android clients.

- `main.cpp`: The core C++ source file containing the Win32 GUI event loop and the `httplib` web server endpoints. It handles uploading APKs, modifying database entries, and serving files to the Android app.
- `db.json`: The database file holding all metadata (App Names, Package Names, Icons, Screenshots, Reviews) for every APK available in the marketplace.
- `apks/` and `images/`: Directories storing the physical `.apk` files and their corresponding icons/screenshots.
- `build.bat`: The primary build script for compiling the server and initiating the full pipeline (detailed below).

B. Android Client (`/Client_App/`)

This is the standard Android Studio project for the "Elite App Marketplace" app. It connects to the C++ Windows Server.

- Built using Java/Android SDK.
- Connects to the local server (via IP address configured by the user) to fetch the JSON app catalog, download APKs, and submit new apps/updates.

- Contains an `UploadActivity.java` which parses metadata (Package Name, Version, App Name) dynamically directly from the selected `.apk` file using Android's native `PackageManager`.

---

3. The Automated Build & Release Pipeline

The project features a **fully automated**, end-to-end continuous integration and deployment pipeline that spans building the C++ server, building the Android APK, signing it, updating the database, pushing to git, and creating a GitHub release.

How to trigger a full build & release: You simply execute `build.bat` in the `Manager_App` directory.

```
cmd.exe /c "build.bat"
```

Step-by-Step Breakdown of the Pipeline:

1. `Manager_App\build.bat` **(The Entry Point):**
    - Kills any running instances of the server.
    - Cleans old binaries.
    - Compiles the C++ source using `g++` and `windres` (statically linked).
    - If the C++ compilation is **successful**, it automatically calls `..\publish_release.ps1` to execute the rest of the pipeline.
2. `publish_release.ps1` **(The Automator):**
    - **Auto-Versioning:** It reads `Manager_App/db.json` to find the latest version of the Elite App Marketplace client and increments the patch number automatically (e.g., `v1.0.53` -> `v1.0.54`).
    - **Android Versioning:** Updates `versionCode` and `versionName` inside `Client_App/app/build.gradle` to match the newly generated version.
    - **Android Compilation:** Triggers `Client_App\build_apk.ps1` to run the Gradle tasks (`assembleDebug` / `assembleRelease`) to compile the `.apk`.
    - **APK Signing:** Uses the `apksigner` build-tool to sign the output APK securely with the `EliteSoftware_Special.pfx` certificate.
    - **Database Injection:** Automatically copies the newly signed APK into the `Manager_App\apks\` folder and updates `Manager_App/db.json` to register the new version. (This allows existing clients to see and download the update immediately).
    - **Git Push:** Stages all changes, commits them as "Auto-build and release v1.0.x", and pushes directly to the `master` branch.
    - **GitHub Release:** Uses the GitHub CLI (`gh release create`) to push a new versioned release, attaching both the Windows `Elite_App_Marketplace-Server.exe` and the `Elite_App_Marketplace-Client_v[version].apk`.
    - **Restart:** Automatically launches the newly compiled Windows Server Executable.

CRITICAL RULE: Never separate the build and publish steps. `build.bat` must always remain the entry point, and it must always seamlessly trigger `publish_release.ps1` upon a successful compile.

Build Logging Details:

- `build.bat` uses a self-logging pattern to prevent it from hanging in the background and keeping the terminal process stuck.
- When executed, the outer script intercepts execution, creates a `build_log.txt` file (wiping the old one cleanly), pipes the entire compilation run into the log file, and immediately calls `exit` when finished to gracefully close the parent process.

4. Android Client Self-Update Architecture

The Elite App Marketplace Android application contains built-in self-updating architecture that must handle edge cases when the Android OS attempts to kill the package while it's being updated.

- **Root/Shizuku Path:** Automated background installations via Shizuku (`pm install`) *must* include the `-r` flag (`pm install -r -S <size>`) so that the package manager knows to overwrite the existing APK, rather than throwing an `INSTALL_FAILED_ALREADY_EXISTS` exception.
- **Standard Fallback Path:** For non-rooted updates, we do not use `Intent.ACTION_VIEW` targeting a local `FileProvider` because Android violently kills the running app during the self-update process, terminating the `FileProvider` mid-stream and corrupting the install. Instead, the application invokes the **PackageInstaller Session API** to stream the raw APK payload directly into the Android OS's staging area *before* committing the install.

Original User Request

2026-08-04T20:28:23-04:00

Teamwork Project Prompt — Draft

> Status: Launched Goal: Craft prompt → get user approval → delegate to teamwork_preview

Fix UI rendering and functional issues in the Local APK Store application, automatically extract and display internal APK icons, and add a connected client list to the server monitor.

Working directory: C:\Users\Administrator\Desktop\Local_APK_Store Integrity mode: development

Requirements

R1. UI Rendering Fixes (Windows App)

Ensure no UI elements have custom backfill colors (they must rely entirely on the OS/Visual Styles, strictly adhering to legacy Windows aesthetics). Fix any overlapping elements and ensure the listview is properly anchored/docked so it resizes dynamically with the window. Do not add any modern UI designs or change the core existing design layout. The Windows app must build upon the existing C++ (or C#/PowerShell depending on existing foundation) codebase without rewriting the underlying structure.

R2. Automatic APK Icon Extraction & Display

Automatically extract the internal icon from APK files and display it within the Windows application's listview. This extraction must be fully integrated into the server and client apps directly, avoiding external tools/binaries unless absolutely required. Ensure this internal icon is also served and displayed correctly on the Android store application's UI, respecting its existing foundation.

R3. Server Monitor Updates

Update the server monitor interface to display a real-time list of connected clients. The list must show both the IP Address and the Device Name of each connected client.

Acceptance Criteria

UI Rendering

- ☐ Programmatic/Visual Verification: No buttons or controls specify a custom background color; they default to OS styling.
- ☐ Programmatic/Visual Verification: No elements overlap when the main window is initialized at default size.
- ☐ Programmatic/Visual Verification: Resizing the window correctly resizes the listview without breaking the layout or obscuring other controls.

APK Icons

- ☐ Programmatic Verification: The server logic successfully reads the internal APK icon for an uploaded/available APK.
- ☐ Visual Verification: The Windows app listview displays the correct internal icon.
- ☐ Visual Verification: The Android store UI successfully fetches and displays the internal APK icon.

Server Monitor

- ☐ Programmatic Verification: When a client connects, the server UI updates to show their IP address and Device Name.
- ☐ Programmatic Verification: Disconnected clients are appropriately managed/removed from the list.

Test Infrastructure & Strategy Documentation — Local APK Store

1. Test Philosophy & Design Principles

The End-to-End (E2E) testing framework for **Local APK Store** is designed around opaque-box, contract-driven, requirement-verified validation.

- **Non-Invasive Verification**: Tests validate observable APIs, network protocols, data structures, and file system states without relying on internal function mocks or fake test assertions.
- **Independence & Isolation**: Every test creates its own temporary environment (isolated ports, isolated temporary directories, synthetic APK archives, isolated mock client connections) and cleans up resources immediately after execution.
- **Strict Verification Rules**: All test cases use authoritative verification derived directly from `ORIGINAL_REQUEST.md` and `PROJECT.md`. No hardcoded dummy passing tests.
- **Multi-Tiered Coverage Layout**: Organizes test suites into Tiers 1-4, moving from individual feature requirement validation up to multi-client end-to-end real-world workflows.

---

2. Feature Inventory & Mapping Matrix

Feature ID	Feature Name	Target Scope / Requirements	Test File

R1	Win32 UI Rendering & Aesthetic Compliance	R1.1 Custom Backfill Brush Removal
		R1.2 Non-Overlapping Control Geometry
		R1.3 SysListView32 Conversion & Dynamic WM_SIZE
		R1.4 Segoe UI Font, Dialogs, Tooltips, Chin Panel, 3D Inset
		R1.5 Log File Creation & Logger Link
			`tests/test_tier1_feature_coverage.py`
			`tests/test_tier2_boundary_corner.py`
			`tests/test_tier3_cross_feature.py`
			`tests/test_tier4_real_world.py`

R2	APK Icon Extraction & Display	R2.1 Server ZIP Internal PNG Extraction
		R2.2 Extraction Fallback (Adaptive XML / Default Image)
		R2.3 HTTP Endpoint `GET /images/<icon>` Serving
		R2.4 Win32 HIMAGELIST Image Loading & ListView Binding
		R2.5 Android Client Intent Extra Alignment & HTTP Icon URL
			`tests/test_tier1_feature_coverage.py`
			`tests/test_tier2_boundary_corner.py`
			`tests/test_tier3_cross_feature.py`
			`tests/test_tier4_real_world.py`

Feature ID	Feature Name	Target Scope / Requirements	Test File

R3	Server Monitor & Connected Clients	R3.1 Client Heartbeat Registration (`POST /api/heartbeat`)
		R3.2 Repeat Heartbeat Timestamp Updates (No Duplicates)
		R3.3 Disconnect Protocol (`POST /api/disconnect`)
		R3.4 15-Second Inactive Client Timeout Purge Thread
		R3.5 Server Monitor SysListView32 UI Timer Refresh
			`tests/test_tier1_feature_coverage.py`
			`tests/test_tier2_boundary_corner.py`
			`tests/test_tier3_cross_feature.py`
			`tests/test_tier4_real_world.py`

---

3. Architecture & Test Directory Structure

```
C:\Users\Administrator\Desktop\Local_APK_Store\
├── tests/
│   ├── __init__.py                          # Package initialization
│   ├── test_tier1_feature_coverage.py        # Tier 1: Feature Coverage (15 tests: R1=5, R2=5, R3=5)
│   ├── test_tier2_boundary_corner.py         # Tier 2: Boundary & Corner Cases (15 tests: R1=5, R2=5, R3=5)
│   ├── test_tier3_cross_feature.py           # Tier 3: Pairwise & Multi-Feature Interactions (4 tests)
│   ├── test_tier4_real_world.py             # Tier 4: Real-World Workflow Scenarios (5 tests)
│   ├── run_e2e_tests.py                     # Native Python Test Runner & Reporter
│   └── run_e2e_tests.ps1                    # PowerShell Test Execution Wrapper
├── TEST_INFRA.md                            # Test Strategy & Infrastructure Manual
└── TEST_READY.md                            # Suite Status, Execution Command & Verification Metrics
```

---

4. Runner Invocation & Commands

PowerShell Invocation (Standard Windows Execution)

```
powershell -ExecutionPolicy Bypass -File tests\run_e2e_tests.ps1
```

Python Direct Invocation

```
python tests/run_e2e_tests.py
```

Individual Tier Execution

```
python -m unittest tests/test_tier1_feature_coverage.py
python -m unittest tests/test_tier2_boundary_corner.py
python -m unittest tests/test_tier3_cross_feature.py
python -m unittest tests/test_tier4_real_world.py
```

5. Coverage Thresholds & Quality Gates

- **Total Test Count Minimum**: 39 Tests
  - Tier 1 (Feature Coverage): >= 15 tests (5 R1, 5 R2, 5 R3)
  - Tier 2 (Boundary & Corner Cases): >= 15 tests (5 R1, 5 R2, 5 R3)
  - Tier 3 (Cross-Feature Pairwise Interactions): >= 4 tests (R1+R2, R2+R3, R1+R3, R1+R2+R3)
  - Tier 4 (Real-World Scenarios): >= 5 tests
- **Pass Threshold**: 100% Pass Rate required (0 Failures, 0 Errors)
- **Execution Code Gate**: Runner exits with code `0` on 100% pass, `1` on any failure.

E2E Test Suite Status & Readiness Report — Local APK Store

STATUS: READY / COMPLETE
SUITE VERIFICATION PASS RATE: 100% (39/39 Tests Passing)
DATE: 2026-08-04

---

1. Test Suite Summary Table

Test Tier	Scope & Focus	Required Tests	Executed Tests	Passed	Failed	Status
Tier 1	Feature Coverage (R1, R2, R3)	>= 15	15	15	0	**PASSED**
Tier 2	Boundary & Corner Cases (R1, R2, R3)	>= 15	15	15	0	**PASSED**
Tier 3	Pairwise & Multi-Feature Interactions	>= 4	4	4	0	**PASSED**
Tier 4	Real-World End-to-End Workflows	>= 5	5	5	0	**PASSED**
TOTAL	**Full Opaque-Box E2E Coverage**	**>= 39**	**39**	**39**	**0**	**PASSED (Code 0)**

---

2. Official Runner Invocation Command

To execute the complete E2E test suite across Tiers 1-4 and verify results:

```
powershell -ExecutionPolicy Bypass -File tests\run_e2e_tests.ps1
```

(Alternatively via Python directly: `python tests/run_e2e_tests.py`)

3. Detailed Feature Verification Checklist

Feature R1: Win32 UI Rendering & Aesthetic Compliance

- ☑ **R1.1 OS Visual Styles & Backfill Brush**: Verified `InitCommonControlsEx`, manifest dependency for Common-Controls 6.0, and `WM_CTLCOLORSTATIC` static background handling defaulting to OS styling.
- ☑ **R1.2 Non-Overlapping Control Geometry**: Verified mathematical bounding box calculations for 850x600 window layout with zero bounding rect intersections.
- ☑ **R1.3 SysListView32 Control & Dynamic WM_SIZE**: Verified listview creation with `SysListView32` class and dynamic `WM_SIZE` client area recalculation using `TCM_ADJUSTRECT`.
- ☑ **R1.4 Segoe UI Font, Dialogs & Aesthetic Compliance**: Verified Segoe UI font initialization (Semibold/Regular), Menubar, Toolbar, About/Help/Settings dialog classes, Tooltips, 3D inset frame, and Chin panel styling.
- ☑ **R1.5 Log File Path & Viewer Launch**: Verified `%SystemDrive%\EliteSoftware\Logs\Manager_App.log` path logger creation, formatting, appending, and log viewer launching.

Feature R2: APK Icon Extraction & Display

- ☑ **R2.1 ZIP Internal PNG Extraction**: Verified extraction of internal icon PNG files from APK archives into `Manager_App/images/<package_name>_icon.png`.
- ☑ **R2.2 XML Adaptive & Missing Icon Fallbacks**: Verified fallback resolution when primary icon is vector XML (`ic_launcher.xml`) or missing, using secondary raster PNG or default fallback image.
- ☑ **R2.3 HTTP `/images/<icon>` Endpoint**: Verified image file serving via HTTP GET `/images/`, returning status 200 OK, `Content-Type: image/png`, and correct image byte stream.
- ☑ **R2.4 Win32 HIMAGELIST & ListView Binding**: Verified GDI+ image loading, HBITMAP conversion, `HIMAGELIST` creation (`ImageList_Create`), and `SysListView32` icon assignment (`LVM_SETIMAGELIST`).
- ☑ **R2.5 Android Client Intent Extra Alignment**: Verified Intent extra parameter alignment (`"app_data"` JSON payload containing `"icon"`, `"name"`, `"package_name"`) in `MainActivity.java` and `AppDetailActivity.java`.

Feature R3: Server Monitor & Connected Clients

- ☑ **R3.1 Heartbeat Endpoint (`POST /api/heartbeat`)**: Verified registration of client IP address, device name, client_id, and last active timestamp in server session map.
- ☑ **R3.2 Repeat Heartbeat Timestamp Updates**: Verified duplicate heartbeats from single client update `last_active` timestamp without creating duplicate client list entries.
- ☑ **R3.3 Disconnect Endpoint (`POST /api/disconnect`)**: Verified immediate removal of client session from server memory database upon disconnect request.
- ☑ **R3.4 15-Second Inactive Timeout Purge**: Verified background cleanup logic purging clients inactive for > 15 seconds.

- [x] **R3.5 Server Monitor SysListView32 UI Refresh**: Verified Server Monitor client list UI (`hwndClientList`) rendering columns (IP Address, Device Name, Last Active) on 1-second `WM_TIMER` tick.

---

4. Test Files Inventory

- `tests/test_tier1_feature_coverage.py`: Tier 1 Feature Unit/Integration Tests
- `tests/test_tier2_boundary_corner.py`: Tier 2 Edge Case & Stress Tests
- `tests/test_tier3_cross_feature.py`: Tier 3 Cross-Feature Pairwise Interaction Tests
- `tests/test_tier4_real_world.py`: Tier 4 Real-World Scenario Workflow Tests
- `tests/run_e2e_tests.py`: Python Execution Runner
- `tests/run_e2e_tests.ps1`: PowerShell Execution Script

Resource Alchemy Hacker - Implementation Plan

This checklist outlines a safe, step-by-step approach to fixing all bugs and implementing new features. As requested, we will execute these items **two at a time**, pausing for testing and feedback before proceeding to the next batch.

Phase 1: Source Code Restoration & UI Stabilization

- [ ] **1. Restore `main.cpp`:** Extract the uncorrupted C++ source code from the local `diff.txt` patch and permanently restore `ResourceAlchemyHacker_GUI/main.cpp`.
- [ ] **2. Fix UI Layout Scaling:** Inject a `WM_SIZE` trigger inside `WinMain` immediately after window creation to ensure the TreeView, Preview Pane, and Status Bar snap into correct alignment on launch without requiring manual resizing.

Phase 2: Text Rendering & Aesthetic Fixes

- [ ] **3. Fix "Version Info" Rendering & Wrapping:** Update the text preview edit control to include `WS_HSCROLL | ES_AUTOHSCROLL`, and ensure extracted UTF-16 text is correctly parsed to avoid mojibake characters.
- [ ] **4. Settings Dialog Aesthetic Overhaul:** Fix the layout clipping of checkboxes in the Settings Dialog and apply the main window's custom gradient title banner logic.

Phase 3: TreeView Previews & Configurable Sizes

- [ ] **5. Dynamic TreeView Icons:** Update the `GuiListResources` routine to dynamically extract `RT_ICON` / `RT_GROUP_ICON` images from the target PE file and bind them to the TreeView's ImageList so each item displays its actual icon.

- ☐ **6. Configurable Icon Sizes:** Add functionality to the Settings Dialog allowing the user to toggle TreeView icon sizes between 16x16, 24x24, and 32x32, persisting the choice.

Phase 4: Safe Extraction & "Demo File" Logic

- ☐ **7. Implement "Demo File!" Button:** Rename the "Test Imageres" button. Implement logic to securely copy `C:\Windows\SystemResources\shell32.dll.mun` to the local working directory and open the copy.
- ☐ **8. Fix "Extract Monolithic" & Context Menus:** Rewrite the extraction logic to iterate through resources, create correctly named sub-folders (e.g., `Icon Groups`, `Bitmaps`), and save them as their native extensions. Add right-click extraction support to the TreeView.

Phase 5: Smart Resource Replacement & File Baking

- ☐ **9. Intelligent Image Conversion:** Implement GDI+ logic in the "Replace Resource" routine so that replacing an Icon Group with a PNG automatically scales it into a multi-resolution `.ico` file, and BMP imports automatically detect and remove pink/gray backgrounds.
- ☐ **10. Auto-Backup System:** Update the save routine to intelligently iterate and generate `.1.bak`, `.2.bak` files for the original executable before writing any modifications.

Phase 6: UX Enhancements & Final Polish

- ☐ **11. Drag and Drop Support:** Add `__argc`/`__argv` parsing to `WinMain` so dragging a PE file onto the executable icon automatically loads the file on startup.
- ☐ **12. File Viewer Tab & Dialog Previews:** Add a new tab/mode to visualize resources in an icon grid (like a file browser) and implement a renderer to preview actual Win32 dialog boxes from `RT_DIALOG` resources.
- ☐ **13. State Persistence:** Save the window position, window size, and a history of the last 30 opened files to the registry or a local config file to be reloaded on launch.
- ☐ **14. Action Menu Wire-Up:** Connect the remaining unimplemented Action Menu and "Add from resource script (.rc)" options to their respective backend logic.

Resource Alchemy Hacker - Investigation Findings

This document outlines the findings from my initial analysis of the project's state, source code, and logic. Our primary goal is to address the bugs and requested features efficiently while maintaining the established C++ Win32 architecture and "EliteSoftware" legacy aesthetics, without destroying or overwriting any functional work.

1. The `main.cpp` Corruption Anomaly

Finding: Upon initial investigation, the physical file at `ResourceAlchemyHacker_GUI/main.cpp` was found to be corrupted, overwritten by a truncated JSON conversation log from a past AI agent's session. **Mitigation Strategy:** Fortunately, I located a complete patch file (`diff.txt`) in the root directory which contains the uncorrupted, 2,897-line source code. I have successfully extracted this source code to a temporary scratch directory. Our first implementation step will be to securely restore this code to `main.cpp` so that normal compilation can resume.

2. Initial Layout Scaling Issue

Finding: The GUI application is a classic C++ Win32 application. When the main window (`g_hwndMain`) is created, Windows automatically sends a `WM_SIZE` message. However, the custom child elements (like the tree view, preview pane, and status bar) are instantiated *after* or *during* this initial layout pass, meaning their internal `WM_SIZE` layout algorithms are never explicitly triggered after all elements exist. **Mitigation Strategy:** Inside `WinMain`, immediately before the `GetMessageW` loop begins, we can inject a manual `PostMessageW(g_hwndMain, WM_SIZE, ...)` or `SendMessageW` to force a complete and correct layout pass after everything has been instantiated. This will instantly snap the UI into place without manual resizing.

3. TreeView Icon Previews

Finding: The `GuiListResources` function parses the PE file but likely assigns a static image index to the `TreeView` nodes. To get Resource Hacker-like previews, the application needs to dynamically extract `RT_ICON` or `RT_GROUP_ICON` resources into an `HICON`, append them to the TreeView's `HIMAGELIST`, and assign that specific image index to the newly created TreeView node. **Mitigation Strategy:** We will modify `GuiListResources` to intercept `RT_ICON` / `RT_GROUP_ICON` iteration. Using Win32 APIs like `CreateIconFromResourceEx`, we can generate the `HICON` dynamically, add it to the TreeView's ImageList (`ImageList_AddIcon`), and set the `iImage` and `iSelectedImage` struct members for the `TVITEM`.

4. "Version Info" Rendering & Line Wrapping

Finding: The "Version Info" resource is typically stored as a `VS_VERSIONINFO` struct containing UTF-16LE text. When extracted, if it's blindly dumped into an ANSI `EDIT` control, you get Mojibake ("盲"?"). Furthermore, Win32 `EDIT` controls word-wrap by default unless explicitly given `WS_HSCROLL` and `ES_AUTOHSCROLL`. **Mitigation Strategy:**

1. Convert the extracted binary buffer from UTF-16 to a standard `std::wstring` and use `SetWindowTextW`.
2. Modify the `CreateWindowEx` call for the text preview pane to include `WS_HSCROLL | ES_AUTOHSCROLL | WS_VSCROLL | ES_MULTILINE`.
3. For line numbers, we will need to inject a custom drawing routine (subclassing the edit control or using a RichEdit control) or a side-panel `STATIC` control synced to the edit control's scrolling.

5. "Extract Monolithic" & Export Logic

Finding: The current export logic seems to just execute a raw file copy or dumping the PE headers, resulting in a broken binary. The goal is to extract resources into folders (e.g., `Export/Icon Groups/`, `Export/Bitmaps/`). **Mitigation Strategy:** We will write a recursive extraction function in the CLI (or GUI) that iterates over `EnumResourceTypesW` and `EnumResourceNamesW`. For each resource, we generate a valid file extension (`.ico`, `.bmp`, `.manifest`) and write the raw binary blob to disk inside a dynamically created directory named after the resource type.

6. Smart File Overwrite & Backups (.bak iteration)

Finding: Saving currently overwrites files blindly or with a hardcoded `_1.bak`. **Mitigation Strategy:** Before writing to the target file, we will implement a loop utilizing `PathFileExistsW` (or `std::filesystem`). It will check for `file.1.bak`, `file.2.bak`, etc., incrementing the counter until a free slot is found, and then execute a safe `CopyFileW` before allowing the modification to occur.

7. Action Menus & Context Menus

Finding: The "Action" menus and "Add from resource script" are disconnected from the command router (`WM_COMMAND`). **Mitigation Strategy:** We will implement `WM_CONTEXTMENU` for the TreeView. Based on the selected node (`TreeView_GetSelection`), we will dynamically populate the Action menu. The "Add from resource script" will invoke `rc.exe` (if available via Windows SDK) or implement a basic resource compiler wrapper.

8. Smart Image Replacement (Format Conversion)

Finding: The user wants to feed the tool any image (PNG, BMP, JPG) and have it automatically converted or scaled to the correct resource type (e.g., building a multi-size `.ico` container from a `.png`). **Mitigation Strategy:** We will utilize GDI+ (`GdiplusStartup`) to load arbitrary image formats. If replacing an Icon Group, we will use GDI+ to rescale the image to `16, 24, 32, 48, 64, 128, 256` sizes and manually construct an `ICONDIR` and `ICONDIRENTRY` structure in memory to inject a perfect ICO file.

Conclusion & Next Steps

None of the planned fixes will destroy existing work. We will proceed methodically, addressing the corruption first, followed by the UI scaling and text rendering issues. We will implement these changes two at a time as requested, ensuring the user can thoroughly test each batch.

Original User Request

Initial Request — 2026-07-10T20:54:36-04:00

Build an automated build, test, and installer system for the Resource Alchemy Hacker project (CLI, GUI, ShellExt). The system must compile the components, run verification tests, and provide a clean installation and uninstallation experience for the end user.

Working directory: C:\Users\Administrator\Desktop\Resource_Alchemy_Hacker\ResourceAlchemyHacker
Integrity mode: development

Requirements

R1. Programmatic Integration Tests

Create an automated script that tests the compiled `ResourceAlchemyHacker_CLI.exe` by extracting a resource from a sample executable and replacing it, verifying the output.

R2. Installer Generation

Create a standard Windows Installer (MSI) or an InnoSetup executable that packages the compiled CLI, GUI, and ShellExt DLL.

R3. Shell Extension Registration

The installer must correctly register the Shell Extension COM DLL (`ResourceAlchemyHacker_ShellExt.dll`) during installation and cleanly unregister it during uninstallation.

Acceptance Criteria

Testing

- ☐ A test script exists and successfully executes the CLI `/extract` command on a target executable, producing a valid output file on disk.
- ☐ The test script executes the CLI `/replace` command and verifies the `_1.bak` backup is created and the target executable is updated.

Installation

- ☐ Running the installer build process produces a single executable setup file (e.g., `.msi` or `.exe`).
- ☐ Installing the setup file copies the CLI, GUI, and DLL to a designated Program Files directory.
- ☐ The installer successfully registers the Shell Extension with the Windows Registry.
- ☐ Running the uninstaller cleanly removes the installed files and unregisters the Shell Extension.

Project: Resource Alchemy Hacker Build, Test, and Installer System

Architecture

Resource Alchemy Hacker consists of three core components:

1. **CLI Engine (`ResourceAlchemyHacker_CLI.exe`)**: Reads, extracts, and replaces resources (like icons, strings, etc.) in PE files.
2. **GUI Client (`ResourceAlchemyHacker_GUI.exe`)**: Classic native Win32 GUI client wrapped in Windows aesthetics.
3. **Shell Extension DLL (`ResourceAlchemyHacker_ShellExt.dll`)**: A COM Shell Extension that provides right-click context menu integration and custom property sheet tabs for PE binaries.

The automated build, test, and installer system needs to:

- Build the solution and all 3 components using MSBuild/Visual Studio.
- Verify CLI features (resource extraction, replacement, and backup creation) via an automated integration test script.
- Build a standard InnoSetup installer executable that installs CLI, GUI, and ShellExt, registers the COM DLL, and cleanly uninstalls everything (including DLL unregistration).

Code Layout

- `/ResourceAlchemyHacker_CLI/` - Source for CLI engine
- `/ResourceAlchemyHacker_GUI/` - Source for GUI app
- `/ResourceAlchemyHacker_ShellExt/` - Source for Shell Extension DLL
- `/tests/` - Directory for test scripts and test binaries (to be created)
- `/Installer/` - InnoSetup script and assets (to be created)

Milestones

#	Name	Scope	Dependencies	Status
1	Explore & Compile	Inspect environment, verify MSBuild/compiler tools, build CLI, GUI, and ShellExt DLL.	None	DONE
2	CLI Integration Tests	Create integration test script and mock binaries to test `/extract`, `/replace`, and backups.	M1	DONE
3	Shell Extension & GUI Compliance	Verify or implement ShellExt self-registration (`DllRegisterServer`/`DllUnregisterServer`) and GUI compliant options.	M1	DONE
4	Installer & COM Registration	Build InnoSetup installer to copy files, register DLL, and cleanly unregister/uninstall.	M2, M3	DONE

#	Name	Scope	Dependencies	Status
5	E2E Testing & Hardening	Complete full E2E testing of build, test, and installation cycle. Run Forensic Auditor.	M4	DONE

Interface Contracts

- **CLI Commands**:
  - `ResourceAlchemyHacker_CLI.exe /list <target_exe>` - Lists resources.
  - `ResourceAlchemyHacker_CLI.exe /extract <target_exe> <type> <name> <lang> <outpath>` - Extracts resource.
  - `ResourceAlchemyHacker_CLI.exe /replace <target_exe> <type> <name> <lang> <inpath>` - Replaces resource, producing backup `<target_exe>_1.bak` or next index.
- **COM registration**:
  - `regsvr32.exe /s ResourceAlchemyHacker_ShellExt.dll` to register context menus and property pages.
  - `regsvr32.exe /u /s ResourceAlchemyHacker_ShellExt.dll` to unregister context menus and property pages.

Resource Alchemy Hacker

EliteSoftwareTech Co. - Antigravity Suite

Description

Resource Alchemy Hacker is a native C++ application for viewing, extracting, and replacing resources in Windows executables (PE files). It serves as a modernized clone of Resource Hacker and includes three main components:

1. **CLI Engine (`ResourceAlchemyHacker_CLI`)**: The backend worker executable that performs all resource extraction and injection using Win32 API.
2. **GUI Application (`ResourceAlchemyHacker_GUI`)**: The frontend Win32 application adhering to EliteSoftware GUI guidelines (Native Win32 aesthetics, Client Edge, Chin, 3D inset, etc.).
3. **Shell Extension (`ResourceAlchemyHacker_ShellExt`)**: A native C++ COM Shell Extension. It provides:
   - Cascading context menus for quick extraction/viewing.
   - A property sheet tab for executable files that lists resources (e.g., icons) like the native icon picker, and allows on-the-fly replacement of resources in system files (using backup renaming strategies).

Project Structure

- `/ResourceAlchemyHacker_CLI/` - Backend command line tool
- `/ResourceAlchemyHacker_GUI/` - Main Win32 application
- `/ResourceAlchemyHacker_ShellExt/` - COM Shell Extension (replaces old SharpShell implementation)
- `/Installer/` - InnoSetup script for deployment

Development Guidelines

- Strictly follows EliteSoftwareTech Co. guidelines for native Win32/WinForms aesthetics.
- Visual Styles must always be explicitly enabled.
- Title banner and 3D Inset Area.
- Dynamic Icon Targeting.

Features (v1.4.0.0 Update)

- **Advanced Resource Previews:**
  - Full Image rendering (Icons, Bitmaps, Cursors, Cursor Groups, Icon Groups).
  - Native AVI and WAV audio playback seamlessly integrated into the GUI.
  - Interactive Text Previews for Manifests, String Tables, Version Info, and Dialogs.
- **Resource Replacement & Extraction:**
  - Intuitive "Replace Resource" dialog with side-by-side graphical previews of original and new resources.
  - Seamless extraction and deletion of resources.
  - Accelerated hotkeys (Ctrl+R) and Context Menu support within the TreeView.
- **Enhanced UI & Aesthetics:**
  - Deep system-level icons dynamically loaded for all specific resource types in the TreeView (Audio, Images, XML, Manifests, Menus).
  - Legacy horizontal scrollbars hidden for a cleaner look (`TVS_NOHSCROLL`).
  - Strict adherence to EliteSoftwareTech Co. guidelines (Client Edge, 3D inset).
  - Registry-based persistence of Window dimensions and settings across sessions.
- **Full Menu Parity:**
  - 25+ classic Resource Hacker functions accessible via native Win32 dropdown menus (`File`, `Edit`, `View`, `Action`, `Help`).

Detailed Component Documentation

1. CLI Engine (`ResourceAlchemyHacker_CLI`)

The CLI is a headless command-line tool that performs resource modifications on PE files. It is designed to be called programmatically by scripts, build tools, or the GUI.

Syntax: `ResourceAlchemyHacker_CLI.exe <action> <target_file> [type] [name] [lang] [file]`

Actions:

- `/list`: Outputs a structured listing of all resources in `<target_file>`.
  - Example: `ResourceAlchemyHacker_CLI.exe /list C:\app.exe`
- `/extract`: Extracts a specific resource to an output file.
  - Example: `ResourceAlchemyHacker_CLI.exe /extract C:\app.exe #14 #1 1033 out.ico`
- `/replace`: Replaces an existing resource with data from `<file>`.
  - Example: `ResourceAlchemyHacker_CLI.exe /replace C:\app.exe #14 #1 1033 new_icon.ico`
- `/add`: Adds a new resource from `<file>`.
  - Example: `ResourceAlchemyHacker_CLI.exe /add C:\app.exe #2 #100 1033 image.bmp`
- `/delete`: Deletes a specific resource.
  - Example: `ResourceAlchemyHacker_CLI.exe /delete C:\app.exe #2 #100 1033`

Note: Types and Names can be strings (e.g. `ICON`) or integers prefixed with `#` (e.g. `#14` for RT_GROUP_ICON). Language should be a numeric LCID (e.g. 1033 for English-US).

2. Shell Extension (`ResourceAlchemyHacker_ShellExt`)

The Shell Extension integrates Resource Alchemy Hacker directly into Windows Explorer using a native C++ COM implementation.

- **Context Menus:** Right-clicking any PE file (.exe, .dll, .sys, etc.) provides quick shortcuts to Extract All Resources or Open in Resource Alchemy Hacker.
- **Property Sheet Tab:** Right-clicking a PE file and selecting "Properties" displays a new "Resources" tab.
  - The tab lists resources in a graphical grid (similar to the Windows icon picker).
  - Users can view and extract resources directly from the properties window without opening the full application.

Important Note on MUI and MUN Files

Modern Windows (Vista and later) frequently uses Multilingual User Interface (MUI) files and MUN files (in `C:\Windows\SystemResources`) to store localized resources (like icons, strings, and dialogs) separate from the main binary.

- When opening a system DLL (e.g. `imageres.dll`), the GUI will display resources dynamically loaded from its associated `.mui` or `.mun` file.
- **Warning:** Saving modifications to the original `.dll` file using this tool **will only write to the `.dll`**. It will **not** modify the `.mui` or `.mun` file. Because of strict core Windows `UpdateResource` API limitations, attempting to modify an icon or resource on a "stub" DLL that forwards to an MUI/MUN file may fail with **Error 50 (ERROR_NOT_SUPPORTED)**.
- **Solution:** To effectively modify system resources and avoid "Error 50", you must explicitly open and edit the actual `.mun` or `.mui` file directly (e.g., `C:\Windows\SystemResources\imageres.dll.mun`), rather than attempting to edit the base stub DLL.

Version

Document Version: 1.3.1.0
